Aula 08

React 

componentes, props e estado

O back-end está pronto 🎉

Uma API com CRUD, camadas, validação e banco.

Falta a tela.

Como vocês faziam no Módulo 0

```
const lista = document.querySelector("#lista");
const contador = document.querySelector("#contador");

function adicionar(texto) {
  const item = document.createElement("li");
  item.textContent = texto;
  lista.appendChild(item);
  contador.textContent = `${lista.children.length} tarefas`;
}
```

O problema

Você precisa lembrar de atualizar

cada pedaço da tela na mão

Com 3 elementos: tranquilo.

Com 30: você esquece um.

E a tela mostra informação errada.

A virada de chave

```
JavaScript puro
  dado mudou → EU atualizo cada pedaço

React
  dado mudou → o REACT atualiza por mim
```

Você descreve **como a tela deve ser**,

não **como mudá-la**.

🚗 Câmbio manual × automático

Manual: você troca cada marcha, na hora certa

Automático: você diz para onde vai,
o carro cuida das marchas

Não é mágica — é uma decisão repetitiva
(que você errava de vez em quando) automatizada.

Criando o projeto

```
npm create vite@latest meu-app -- --template react
cd meu-app
npm install
npm run dev
```

Abre em **http://localhost:5173**

⚠️ Duas portas agora

Back-end	`localhost:3000`
Front-end	`localhost:5173`

São **dois programas separados**.

Na aula 09 eles vão conversar.

A estrutura

```
meu-app/
├── index.html      só uma <div id="root"> vazia
└── src/
    ├── main.jsx    liga o React ao HTML
    ├── App.jsx     componente principal
    ├── App.css
    └── components/ ← você cria
```

Primeira faxina: apague o conteúdo de `App.jsx` e `App.css`. Comece do zero.

Componente = função que devolve tela

```
function Saudacao() {
  return <h1>Olá, turma FPA!</h1>;
}
```

Duas regras

1. Nome com **letra maiúscula**
2. Devolve **um** elemento raiz (use `<>...</>`)

Usa-se como uma tag

```
function App() {
  return (
    <div>
      <Saudacao />
      <Saudacao />
    </div>
  );
}
```

JSX: HTML dentro do JavaScript

```
const nome = "Maria";
const idade = 28;

<div className="perfil">
  <h2>{nome}</h2>
  <p>Nasceu em {2026 - idade}</p>
  <p>{idade >= 18 ? "Maior" : "Menor"}</p>
</div>
```

`{}` = "aqui entra JavaScript"

Dentro das chaves só cabe *expressão*

✅ variável, conta, ternário, `.map()`

❌ `if`, `for`

Por isso usamos ternário e `map` o tempo todo.

JSX ≠ HTML

HTML	JSX
`class`	**`className`**
`for`	`htmlFor`
`onclick="..."`	`onClick={fn}`
` `	` `
`style="color:red"`	`style={{ color: "red" }}`

O erro nº 1: `class` em vez de `className`

Renderização condicional

```
{carregando && <p>Carregando...</p>}

{erro ? <p>{erro}</p> : <Lista />}
```

⚠️ `{lista.length && <Lista />}`

→ imprime "**0**" na tela quando vazio!

Use `{lista.length > 0 && <Lista />}`

Props = parâmetros do componente

```
function Card({ titulo, descricao }) {
  return (
    <div className="card">
      <h3>{titulo}</h3>
      <p>{descricao}</p>
    </div>
  );
}

<Card titulo="React" descricao="Interfaces" />
```

Aspas ou chaves?

```
<Card
  titulo="texto"          {/* string: aspas */}
  idade={28}              {/* número: chaves */}
  ativo={true}            {/* boolean: chaves */}
  itens={[1, 2]}          {/* array: chaves */}
  aoClicar={minhaFn}      {/* função: chaves */}
/>
```

Só string usa aspas. Todo o resto: chaves.

⚠️ Props são somente leitura

Um componente **nunca** altera as props que recebeu.

Precisa mudar algo?

→ ou é **estado**

→ ou é trabalho de quem passou a prop

children

```
function Painel({ titulo, children }) {
  return (
    <section>
      <h2>{titulo}</h2>
      {children}
    </section>
  );
}

<Painel titulo="Tarefas">
  <p>Isto chega como children</p>
</Painel>
```

`useState`: o que muda com o tempo

```
import { useState } from "react";

function Contador() {
  const [contagem, setContagem] = useState(0);
  //       ↑ valor    ↑ função       ↑ inicial

  return (
    <button onClick={() => setContagem(contagem + 1)}>
      Cliquei {contagem} vezes
    </button>
  );
}
```

O que acontece no clique

```
1. onClick chama setContagem(1)
2. o React guarda o novo valor
3. o React chama Contador() DE NOVO
4. agora contagem vale 1 → o JSX sai diferente
5. o React atualiza só o que mudou na tela
```

O componente inteiro roda de novo.

É isso que torna tudo previsível.

Regra 1 — nunca altere direto

```
contagem = contagem + 1;       // ❌ tela não muda
setContagem(contagem + 1);     // ✅
```

Regra 2 — objetos e arrays: valor NOVO

```
tarefas.push(nova);                    // ❌
setTarefas([...tarefas, nova]);        // ✅

usuario.nome = "Ana";                          // ❌
setUsuario({ ...usuario, nome: "Ana" });       // ✅
```

🔴 O erro nº 1 de quem começa

> "Mudei o array e a tela não atualiza!"

O React compara **referências**.

Mesmo array na memória = "nada mudou".

Aquela aula sobre spread era sobre isto.

Regra 3 — hooks só no topo

```
if (algo) {
  const [x, setX] = useState(0);   // ❌ quebra
}
```

Nunca dentro de `if`, `for` ou outra função.

Listas com `.map()`

```
<ul>
  {tarefas.map((tarefa) => (
    <li key={tarefa.id}>{tarefa.titulo}</li>
  ))}
</ul>
```

O mesmo `.map()` da aula 02 —
agora transformando dados em **tela**.

🔧 Por que `.map()` e não `for`?

Porque dentro do JSX só cabe **expressão**

(algo que resulta num valor).

```
{ for (...) { ... } }          // ❌ não é expressão
{ lista.map(...) }             // ✅ resulta num array
```

O React sabe desenhar um **array de elementos**.

É por isso que `map` é o método do React.

Por que `key`?

É a **identidade** de cada item.

Sem ela o React não sabe qual item é qual.

✅ o `id` que vem do banco

❌ o índice do array

Com índice, reordenou ou removeu do meio

→ o React associa o item errado → bugs bizarros

Eventos

```
<button onClick={aoClicar}>        {/* ✅ */}
<button onClick={aoClicar()}>      {/* ❌ chama na hora! */}
<button onClick={() => remover(id)}>  {/* ✅ com argumento */}
```

Sem a arrow, a função roda ao **desenhar**,

não ao clicar. Se mexer no estado: loop infinito.

Recapitulando

1. Componente devolve tela · props entram de fora e são só leitura
2. `useState` guarda o que muda — e mudar redesenha a tela
3. **Nunca mute o estado** — crie um valor novo

Mão na massa 🚀

Próxima aula: o React vai buscar dados na **sua API** 🔌